Payment Form QA Assessment

Risk-Based Analysis | Test Cases | Recommended Mitigation

Scope: Review of the provided payment form mockup. No functional application or backend implementation was supplied for verification, Leon Markovich.

Part A. Potential Issues and Risk Assessment

Note

Severity levels are estimated based on the potential impact on users, payment functionality, data security, and business operations.

Priority is assessed based on severity, likelihood of occurrence, business impact, and urgency.

Since this assessment is based on a mockup rather than a functioning application, all ratings are preliminary and should be reviewed after functional testing and clarification of business requirements.

A1. Observable usability concerns

Issue	Priority	Observation / Impact
Missing currency	High	The amount appears as “30.00” without a currency code or symbol; customers cannot confirm what they will be charged.
Unclear payment frequency	High	The form does not explain whether the charge is one-time or recurring.
Ambiguous Continue action	Medium	It is unclear whether Continue charges the card or opens a review step.
Address line 2 not labeled optional	Low	Users may unnecessarily complete this field.
CVV/CVC not visible	Needs review	Some payment flows do not require CVV. Confirm requirements with the payment provider rather than treating its absence as a defect.
Middle Initial (MI) Field Not Clearly Marked as Optional	Low	The MI field may confuse users who do not have a middle name. The form should clearly indicate that this field is optional.

A2. Functional and reliability risks requiring testing

Risk	Impact if confirmed	Verification
Duplicate payment submission	Critical	Rapid repeated clicks, retries and timeouts must not create duplicate charges.
Invalid or expired card accepted	High	Verify format/checksum checks, expiration validation and server-side enforcement.
Required fields bypassed	Medium	Verify missing or malformed inputs cannot progress.
Poor processing feedback	Medium	Check loading states, clear outcomes and recovery from slow or failed requests.

A3. Security risks requiring implementation-level verification

Risk	Impact if confirmed	Verification
Sensitive card data exposure	Critical	Inspect transmission, logging and storage; verify use of a PCI DSS-compliant payment integration.
Missing server-side validation	High	Verify malformed or manipulated requests are rejected server-side.
Missing 3D Secure support where required	High	Verify 3DS challenge/frictionless flows and successful, failed or canceled authentication outcomes when applicable.

Important: 3D Secure is not synonymous with requiring a visible two-factor prompt for every payment. Authentication may be frictionless, and requirements vary by issuer, payment provider and jurisdiction.

Risk prioritization

Prioritize protection of sensitive card data and prevention of duplicate charges, followed by clear payment amount/currency and recurring-payment disclosure. The latter concerns are visible in the mockup; the former must be tested.

Part B. Test Cases

TC-01: Valid Payment Information

Priority: High | Type: Functional / Positive

Objective: Confirm that a correctly completed form permits the expected next step.

Preconditions:

- The payment form is accessible.
- Approved sandbox card details and test environment are available.

Steps:

1. Open the form and select VISA.
2. Enter a valid sandbox VISA card number and cardholder name.
3. Select a future expiration date.
4. Complete all mandatory billing fields.
5. Click Continue.

Expected results:

- All valid inputs are accepted without validation errors.
- The user advances to the expected next stage.

- If Continue processes payment, the sandbox transaction follows the documented success flow.

TC-02: Invalid Card and Expiration Validation

Priority: High | Type: Functional / Negative

Objective: Verify that invalid card details are rejected with actionable feedback.

Preconditions:

- The payment form is accessible.
- All fields not under test contain valid sandbox values.

Steps:

1. Select VISA and enter a card number that fails checksum validation.
2. Complete other required fields and click Continue.
3. Record the validation result.
4. Replace the number with a valid sandbox card number.
5. Choose an expired date if the UI allows it, then click Continue.

Expected results:

- Invalid card data is rejected and a clear error appears.
- Expired dates cannot be selected or are rejected.
- Invalid requests cannot advance or initiate a successful charge.
- Server-side validation rejects invalid inputs where applicable.

TC-03: Duplicate Submission Prevention

Priority: Critical | Type: Reliability / Negative

Objective: Ensure one intended payment operation cannot result in multiple successful charges.

Preconditions:

- Valid sandbox payment details are available.
- Payment requests and transaction records can be inspected.
- The tested step actually initiates payment processing.

Steps:

1. Complete the form with valid sandbox details.
2. Trigger the payment action repeatedly in rapid succession.
3. Observe button state, progress indicators and requests.
4. Inspect sandbox transaction records.
5. If supported, simulate a timeout and retry the same operation.

Expected results:

- The UI provides clear processing feedback and prevents accidental repeat actions.
- No more than one successful charge is created for the same intended payment operation.
- Retries are handled safely and the final payment status is accurately displayed.

Part C. Proposed Solution: Prevent Duplicate Charges

Selected risk: Duplicate payment processing (Critical if confirmed). A UI-only fix is insufficient because retries, concurrent requests and network failures can still generate duplicate requests.

Recommended mitigation

Generate a unique idempotency key for each intended payment operation and reuse it for retries of that same operation.
Enforce idempotency on the backend/payment-provider boundary: atomically associate the key with the payment result, so concurrent equivalent requests cannot create additional charges.
Disable the payment action while processing and display a loading indicator and clear status.
On timeout or uncertain outcome, query the existing payment status rather than automatically creating a new charge.

Log payment IDs, idempotency keys and status transitions without logging full card details.

Acceptance criteria

- Repeated clicks and retries with the same payment-operation key yield at most one successful charge.
- Concurrent requests do not bypass backend deduplication.
- The UI displays an accurate outcome after success, failure or network interruption.
- A genuinely new payment operation can be submitted with a new key.

Assessment note: All risks described as potential must be validated against actual product requirements and implementation before being reported as confirmed defects.